BIG DATA TECHNOLOGY LAB (BDTL) · PRACTICAL REPORT

Apache Hive Experiments 1 and 4

Submitted by

Adesh Patankar

Experiment	Title and learning objective
Experiment 1	**Create and Describe Tables in Hive** Create a managed and external table using different file formats (TEXTFILE, ORC, PARQUET). Verify structure using DESCRIBE and SHOW TABLES.
Experiment 4	**Alter and Drop Table Operations** Practice schema evolution with ALTER TABLE (rename, add/replace columns, change data type). Drop and recover tables using DROP and MSCK REPAIR TABLE.

Platform: Apache Hive 3.1.2 on Apache Hadoop 3.3.6 (pseudo-distributed HDFS + YARN), Ubuntu 24.04 LTS. All execution evidence in this report consists of screenshots of the actual terminal session in which the experiments were run.

Contents

	Section
Experiment 1	**Create and Describe Tables in Hive**
	A. Experiment Details B. Aim C. Theory D. Requirements E. Procedure and Implementation F. Execution Screenshots G. Output and Results H. Result I. Observations and Troubleshooting
Experiment 4	**Alter and Drop Table Operations**
	A. Experiment Details B. Aim C. Theory D. Requirements E. Procedure and Implementation F. Execution Screenshots G. Output and Results H. Result I. Observations and Troubleshooting

Experiment 1: Create and Describe Tables in Hive

A. Experiment Details

Field	Details
Sub-experiment No.	1
Title	Create and Describe Tables in Hive
Description / Learning Objective (bucket list)	Create a managed and external table using different file formats (TEXTFILE, ORC, PARQUET). Verify structure using DESCRIBE and SHOW TABLES.
Student	Adesh Patankar
Subject	Big Data Technology Lab (BDTL)
Tool / Technology	Apache Hive 3.1.2 on Apache Hadoop 3.3.6 (HDFS + YARN)

B. Aim

To create **managed (internal)** and **external** Hive tables stored in three different file formats – **TEXTFILE**, **ORC** and **PARQUET** – load the same employee dataset into each of them, and verify their structure and metadata using `SHOW TABLES`, `DESCRIBE` and `DESCRIBE FORMATTED`.

C. Theory

C.1 Apache Hive

Apache Hive is a data-warehouse system built on top of Hadoop. It lets users describe data that is stored in HDFS as *tables* and query it with **HiveQL**, a SQL-like language. Hive follows a **schema-on-read** model: the table definition (columns, types, file format, location) is stored in the **Metastore**, while the data itself stays as files in HDFS. When a query is submitted, the Hive driver parses and compiles it, looks up the table metadata in the Metastore, and the execution engine (MapReduce in this setup) runs the resulting jobs on YARN.

C.2 Managed vs External tables

Aspect	Managed (internal) table	External table
Created with	`CREATE TABLE ...`	`CREATE EXTERNAL TABLE ... LOCATION '<hdfs path>'`
Data location	Inside the warehouse: `/user/hive/warehouse/<db>.db/<table>`	Any HDFS directory chosen by the user
Ownership of data	Hive owns both metadata and data files	Hive owns only the metadata; the files belong to the user / other tools
`DROP TABLE`	Deletes metadata **and** the data files	Deletes only the metadata; files remain in HDFS
Table Type shown by DESCRIBE FORMATTED	`MANAGED_TABLE`	`EXTERNAL_TABLE` (and table parameter `EXTERNAL=TRUE`)
Typical use	Intermediate / Hive-only data	Shared raw data, data produced by other systems

C.3 File formats

Format	Layout	Key characteristics	Hive classes (SerDe / InputFormat)
TEXTFILE	Row-oriented plain text	Human-readable; delimiter defined by `ROW FORMAT DELIMITED FIELDS TERMINATED BY`; no compression or indexes by default; can be loaded directly with `LOAD DATA`.	`LazySimpleSerDe` / `TextInputFormat`
ORC (Optimized Row Columnar)	Columnar, stripes + footer	Stores each column together with lightweight indexes and statistics (min/max, counts); good compression and predicate push-down; required for Hive ACID tables. Files start with the magic bytes `ORC`.	`OrcSerde` / `OrcInputFormat`
PARQUET	Columnar, row groups + footer	Open columnar format shared by Spark, Impala, Drill etc.; efficient column pruning and encoding. Files start and end with the magic bytes `PAR1`.	`ParquetHiveSerDe` / `MapredParquetInputFormat`

Because ORC and PARQUET are binary formats, a plain CSV file cannot simply be moved into such a table. The usual pattern, used in this experiment, is to load the CSV into a TEXTFILE table first and then convert it with `INSERT OVERWRITE TABLE <orc/parquet table> SELECT * FROM <text table>`, which runs a MapReduce job that rewrites the rows in the target format.

C.4 Commands used to verify tables

- `SHOW TABLES [LIKE 'pattern']` – lists the tables of the current database (wildcard `*` allowed).
- `DESCRIBE table` – shows column names and data types.
- `DESCRIBE FORMATTED table` – shows the full metadata: owner, location, table type, table parameters (row count, size, comment, EXTERNAL flag) and storage information (SerDe, input/output format, buckets).
- `DESCRIBE DATABASE db` – shows the comment, HDFS location and owner of a database.

D. Requirements

Item	Used in this experiment
Host machine	Apple MacBook Air (Apple Silicon, arm64) running macOS
Virtual machine	Multipass VM `bigdata` – Ubuntu 24.04 LTS (arm64), 2 vCPU, 4 GB RAM, 20 GB disk, 2 GB swap
Java	OpenJDK 8 (`/usr/lib/jvm/java-8-openjdk-arm64`)
Hadoop	Apache Hadoop 3.3.6, pseudo-distributed (single node): HDFS at `hdfs://localhost:9000` + YARN; daemons NameNode, DataNode, SecondaryNameNode, ResourceManager, NodeManager
Hive	Apache Hive 3.1.2, Hive CLI, execution engine MapReduce (Hive-on-MR), embedded Apache Derby metastore, warehouse `/user/hive/warehouse`
Scripts	Bash runner scripts + HiveQL files (supplied separately in *Scripts/*); each HiveQL file is run with `hive -v -f` so that every statement is echoed before its output
Dataset	`employees.csv` – 8 comma-separated records (emp_id, name, department, salary, city)
HDFS paths	Managed tables: `/user/hive/warehouse/exp1_db.db/`; external tables: `/bdt/exp1/ext_text`, `/bdt/exp1/ext_orc`, `/bdt/exp1/ext_parquet`
Pre-requisites	Hadoop daemons running (`start-dfs.sh`, `start-yarn.sh`), Hive metastore initialised (`schematool -dbType derby -initSchema`), warehouse and `/tmp` directories created in HDFS

E. Procedure and Implementation

The experiment was executed with the runner script `bdt/exp1/run_exp1.sh`. The script first resets the experiment (drops `exp1_db` and `/bdt/exp1`) so it can be re-run, then executes eight parts. For every HiveQL part it runs `hive -v --hivevar datadir=... -f <file>.sql`; the `-v` (verbose) flag prints each statement before its result, so every screenshot shows the code together with its execution log and output.

Step 1 – Input dataset and upload to HDFS

exp1/data/employees.csv

```
101,Aarav Shah,IT,65000,Pune
102,Diya Mehta,HR,48000,Mumbai
103,Kabir Rao,Finance,72000,Bengaluru
104,Meera Iyer,IT,81000,Chennai
105,Rohan Das,Sales,39000,Kolkata
106,Sneha Kulkarni,Marketing,52000,Pune
107,Arjun Nair,Finance,58000,Kochi
108,Isha Verma,Sales,45000,Delhi
```

Shell commands

```
cat data/employees.csv
hdfs dfs -mkdir -p /bdt/exp1/ext_text
hdfs dfs -put -f data/employees.csv /bdt/exp1/ext_text/
```

Step 2 – Create the database and the MANAGED tables (TEXTFILE, ORC, PARQUET)

The TEXTFILE table is filled with `LOAD DATA LOCAL INPATH` (the file is copied into the warehouse directory). The ORC and PARQUET tables are filled with `INSERT OVERWRITE ... SELECT`, which converts the rows into the columnar formats.

exp1/sql/01_database_and_managed_tables.sql

```
-- Experiment 1 | Step 1 : create a database for this experiment
CREATE DATABASE IF NOT EXISTS exp1_db
COMMENT 'BDT Lab Experiment 1 - create and describe tables';

SHOW DATABASES;
DESCRIBE DATABASE exp1_db;

-- Experiment 1 | Step 2 : MANAGED (internal) tables in three file formats
USE exp1_db;

-- 2a. Managed table stored as TEXTFILE (comma-delimited)
CREATE TABLE IF NOT EXISTS emp_text (
  emp_id     INT,
  name       STRING,
  department STRING,
  salary     DOUBLE,
  city       STRING)
COMMENT 'Managed table - TEXTFILE'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;

LOAD DATA LOCAL INPATH '${hivevar:datadir}/employees.csv' OVERWRITE INTO TABLE emp_text;

-- 2b. Managed table stored as ORC (columnar, compressed)
CREATE TABLE IF NOT EXISTS emp_orc (
  emp_id INT, name STRING, department STRING, salary DOUBLE, city STRING)
COMMENT 'Managed table - ORC'
STORED AS ORC;

INSERT OVERWRITE TABLE emp_orc SELECT * FROM emp_text;

-- 2c. Managed table stored as PARQUET (columnar)
CREATE TABLE IF NOT EXISTS emp_parquet (
  emp_id INT, name STRING, department STRING, salary DOUBLE, city STRING)
COMMENT 'Managed table - PARQUET'
STORED AS PARQUET;

INSERT OVERWRITE TABLE emp_parquet SELECT * FROM emp_text;
```

Step 3 – Create the EXTERNAL tables (TEXTFILE, ORC, PARQUET)

The external TEXTFILE table is defined directly over the CSV uploaded to `/bdt/exp1/ext_text` in Step 1 – no data is loaded, Hive simply reads the files already present at the LOCATION. The external ORC and PARQUET tables are populated from it.

exp1/sql/03_external_tables.sql

```
-- Experiment 1 | Step 3 : EXTERNAL tables in three file formats
-- (data lives at the LOCATION we give; Hive only keeps the metadata)
USE exp1_db;

-- 3a. External TEXTFILE table over a CSV already uploaded to HDFS
CREATE EXTERNAL TABLE IF NOT EXISTS emp_ext_text (
  emp_id INT, name STRING, department STRING, salary DOUBLE, city STRING)
COMMENT 'External table - TEXTFILE'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE
```

```
LOCATION '/bdt/exp1/ext_text';

-- 3b. External ORC table
CREATE EXTERNAL TABLE IF NOT EXISTS emp_ext_orc (
  emp_id INT, name STRING, department STRING, salary DOUBLE, city STRING)
COMMENT 'External table - ORC'
STORED AS ORC
LOCATION '/bdt/exp1/ext_orc';

INSERT OVERWRITE TABLE emp_ext_orc SELECT * FROM emp_ext_text;

-- 3c. External PARQUET table
CREATE EXTERNAL TABLE IF NOT EXISTS emp_ext_parquet (
  emp_id INT, name STRING, department STRING, salary DOUBLE, city STRING)
COMMENT 'External table - PARQUET'
STORED AS PARQUET
LOCATION '/bdt/exp1/ext_parquet';

INSERT OVERWRITE TABLE emp_ext_parquet SELECT * FROM emp_ext_text;
```

Step 4 – Verify with SHOW TABLES and DESCRIBE

exp1/sql/04_show_and_describe.sql

```
-- Experiment 1 | Step 4 : verify the tables with SHOW TABLES and DESCRIBE
USE exp1_db;

SHOW TABLES;
SHOW TABLES LIKE 'emp_ext*';

DESCRIBE emp_text;
DESCRIBE emp_orc;
DESCRIBE emp_ext_parquet;
```

Step 5 – Inspect full metadata with DESCRIBE FORMATTED

exp1/sql/05_describe_formatted_managed.sql

```
-- Experiment 1 | Step 5 : full metadata of a MANAGED table (ORC)
-- Look for: Table Type = MANAGED_TABLE, Location under /user/hive/warehouse,
--           InputFormat / SerDe = ORC
USE exp1_db;
DESCRIBE FORMATTED emp_orc;
```

exp1/sql/06_describe_formatted_external.sql

```
-- Experiment 1 | Step 6 : full metadata of an EXTERNAL table (PARQUET)
-- Look for: Table Type = EXTERNAL_TABLE, EXTERNAL = TRUE, Location = /bdt/exp1/ext_parquet,
--           InputFormat / SerDe = Parquet
USE exp1_db;
DESCRIBE FORMATTED emp_ext_parquet;
```

Step 6 – Query every format

exp1/sql/07_query_all_tables.sql

```
-- Experiment 1 | Step 7 : the same data can be read from every format
USE exp1_db;

SELECT * FROM emp_text;
SELECT * FROM emp_orc WHERE department = 'IT';
SELECT * FROM emp_ext_parquet WHERE salary > 55000;
SELECT department, COUNT(*) AS employees, AVG(salary) AS avg_salary
FROM emp_ext_orc GROUP BY department;
```

Step 7 – Inspect the files in HDFS and prove the file formats

Shell commands

```
hdfs dfs -ls -R /user/hive/warehouse/exp1_db.db      # managed tables
hdfs dfs -ls -R /bdt/exp1                           # external tables
# first 4 bytes of one data file of each managed table
hdfs dfs -cat /user/hive/warehouse/exp1_db.db/emp_text/employees.csv | head -c 4; echo
hdfs dfs -cat /user/hive/warehouse/exp1_db.db/emp_orc/000000_0      | head -c 4; echo
hdfs dfs -cat /user/hive/warehouse/exp1_db.db/emp_parquet/000000_0  | head -c 4; echo
```

F. Execution Screenshots

All screenshots below were captured from the Terminal of the Ubuntu VM (`ubuntu@bigdata`) while `run_exp1.sh` was executing. Only empty margins were trimmed for printing; no terminal content was edited.

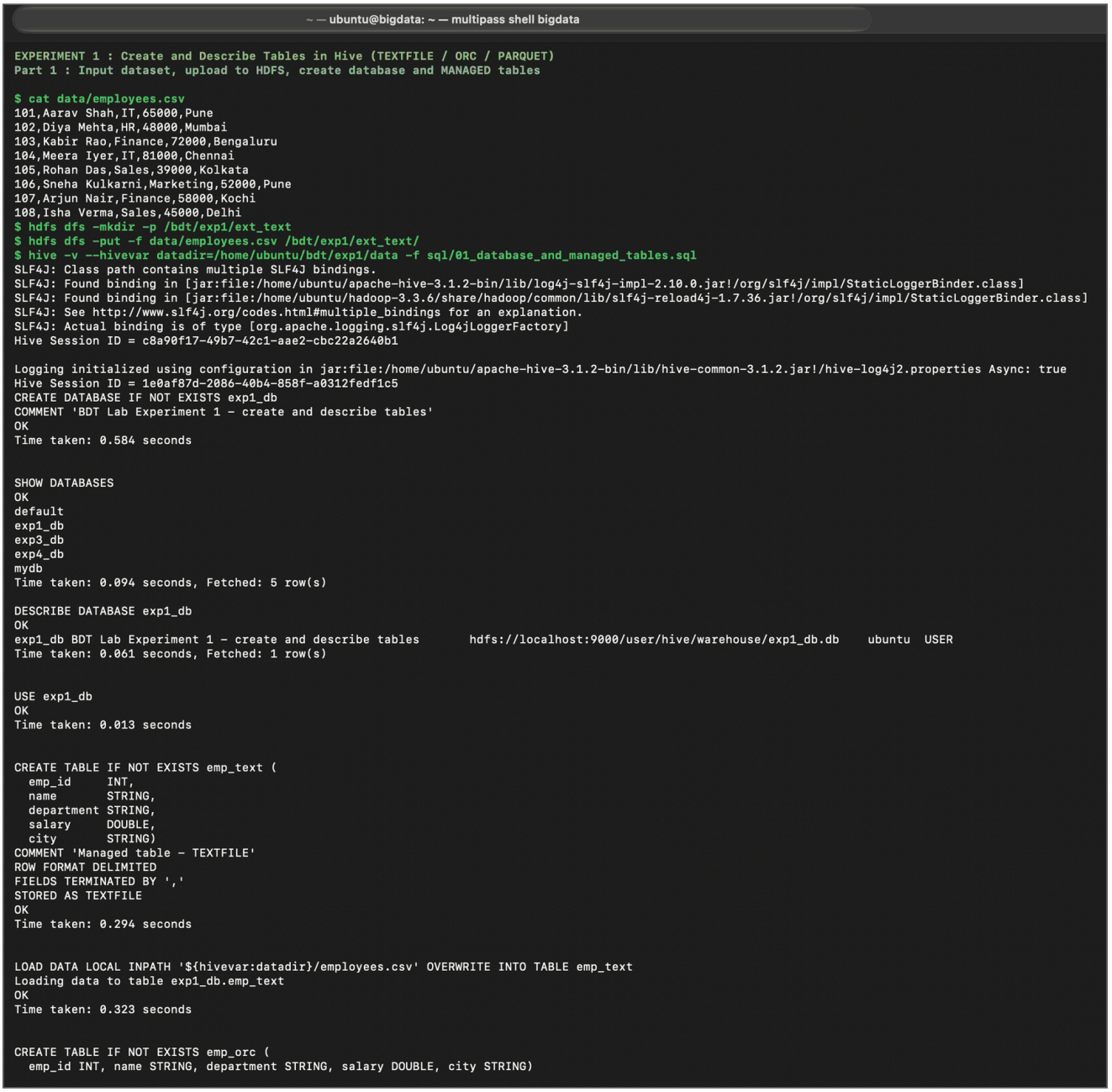


Figure 1.1: Input dataset, HDFS upload, database creation and the managed TEXTFILE table

The 8-row `employees.csv` is displayed and uploaded to `/bdt/exp1/ext_text`. Hive then creates `exp1_db` (`DESCRIBE DATABASE` shows its warehouse location `hdfs://localhost:9000/user/hive/warehouse/exp1_db.db`), creates the managed table `emp_text` (STORED AS TEXTFILE) and loads the CSV with `LOAD DATA LOCAL INPATH`.

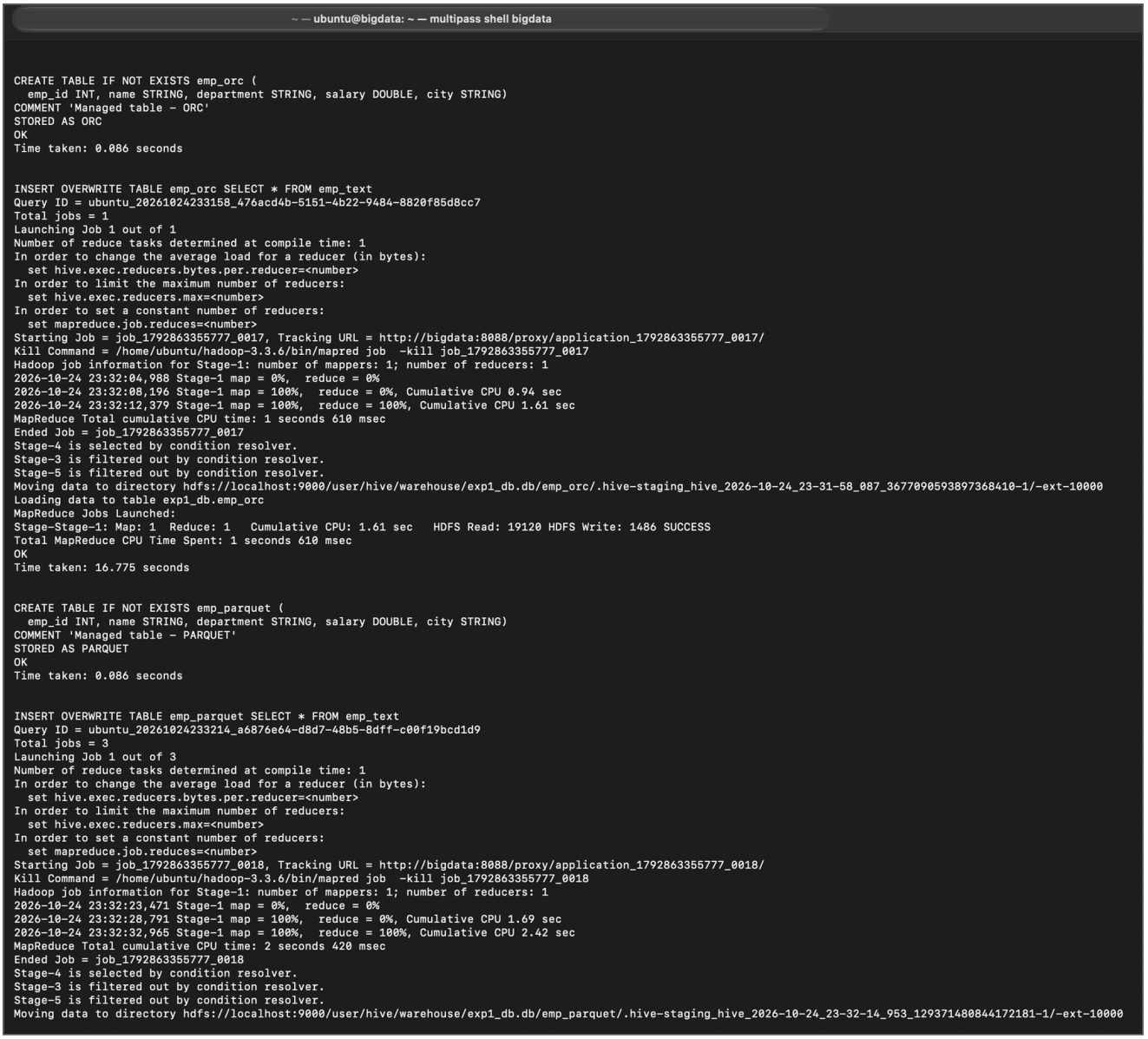


Figure 1.2: Managed ORC and PARQUET tables created and loaded through MapReduce jobs

`emp_orc` (STORED AS ORC) and `emp_parquet` (STORED AS PARQUET) are created. Each `INSERT OVERWRITE ... SELECT * FROM emp_text` launches a MapReduce job on YARN (job IDs `..._0017` and `..._0018`); the ORC job finishes with `SUCCESS` and the data is moved into the table directory under the warehouse.

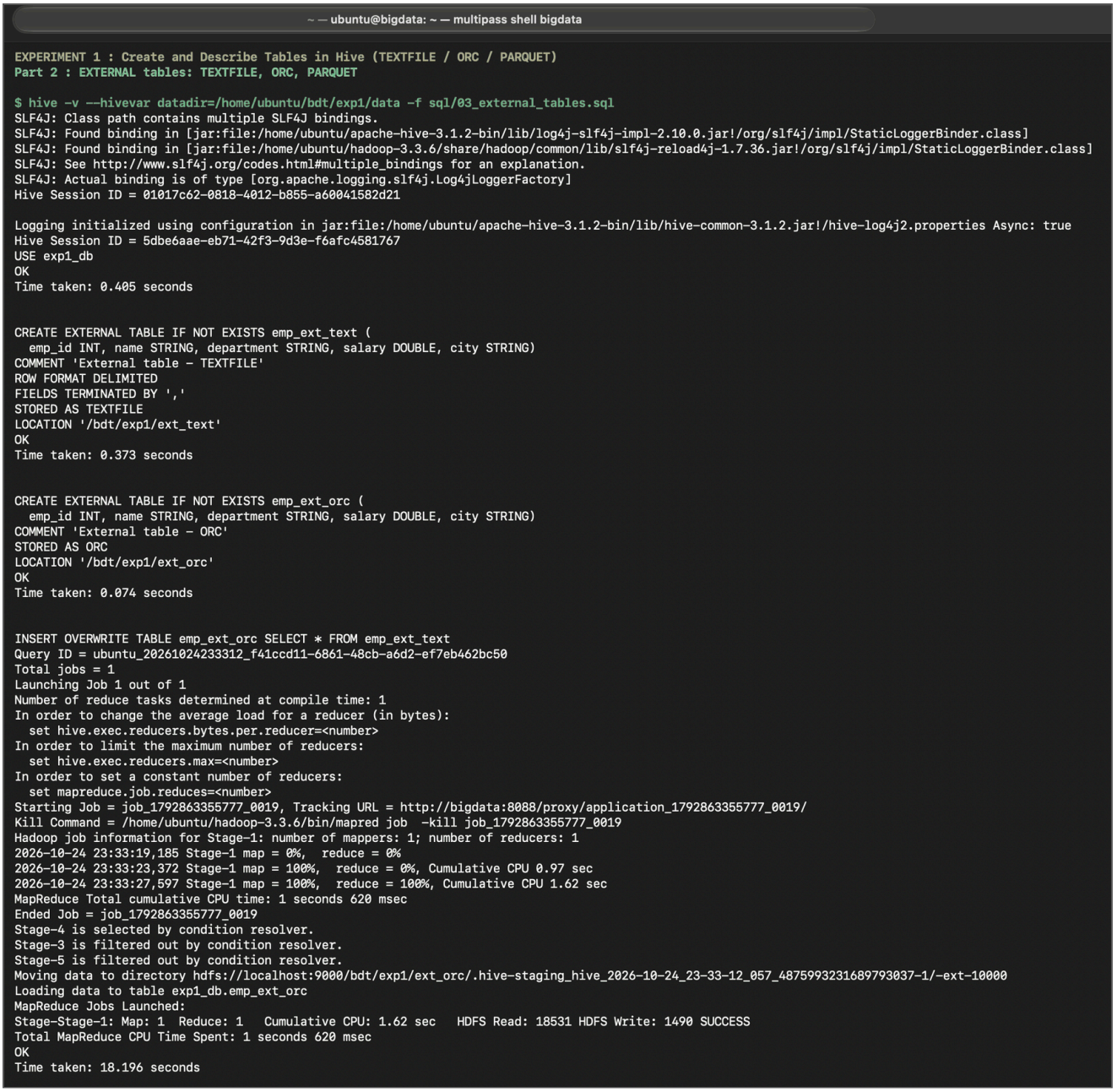


Figure 1.3: External TEXTFILE and ORC tables with their own HDFS LOCATION

`emp_ext_text` is defined over the existing directory `/bdt/exp1/ext_text`; `emp_ext_orc` points to `/bdt/exp1/ext_orc` and is populated by a MapReduce job that ends with `SUCCESS`.

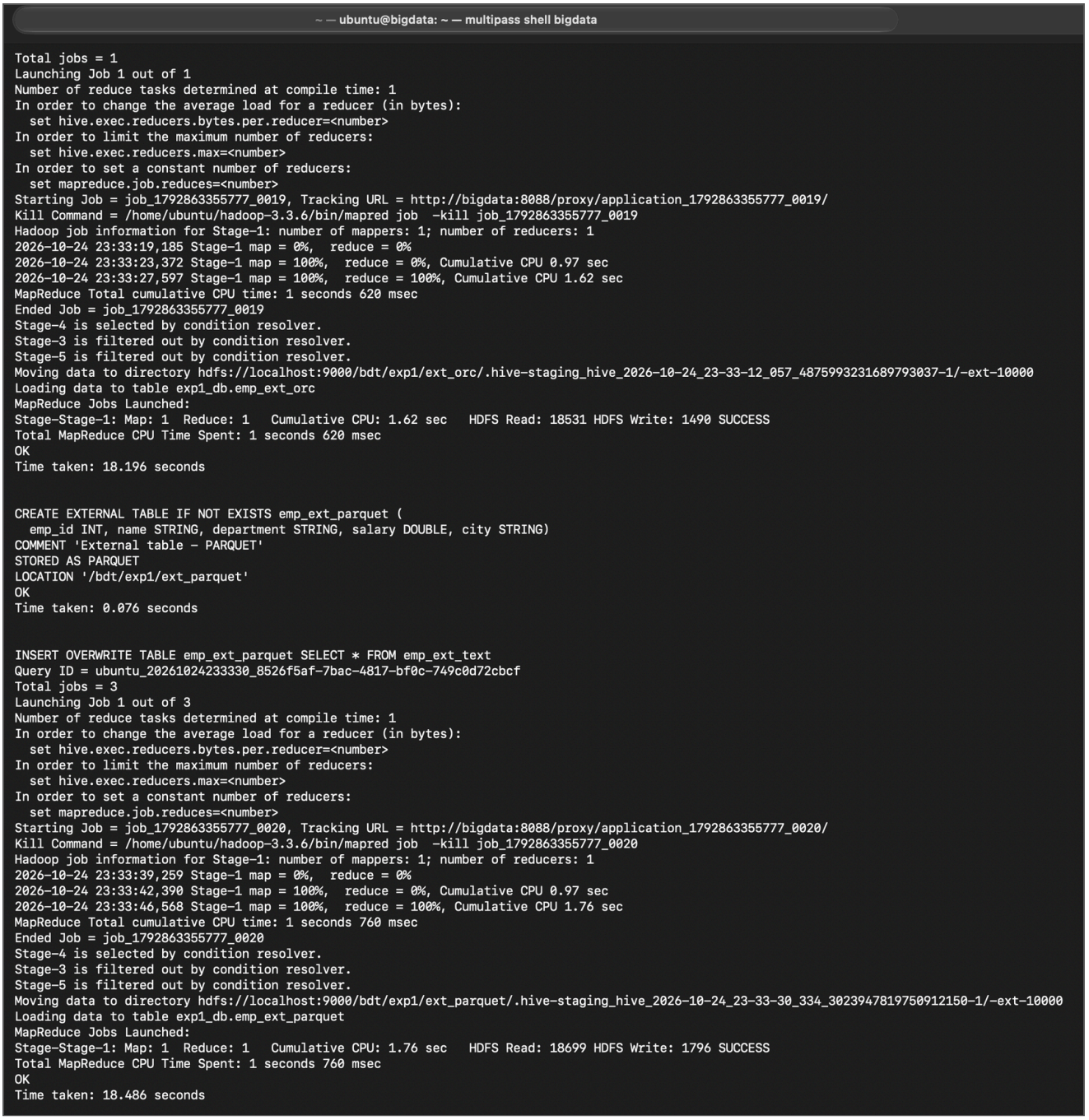


Figure 1.4: External PARQUET table created and populated

`emp_ext_parquet` (STORED AS PARQUET, LOCATION `/bdt/exp1/ext_parquet`) is created and loaded from the external text table; the job log shows `Loading data to table exp1_db.emp_ext_parquet` and `SUCCESS`.

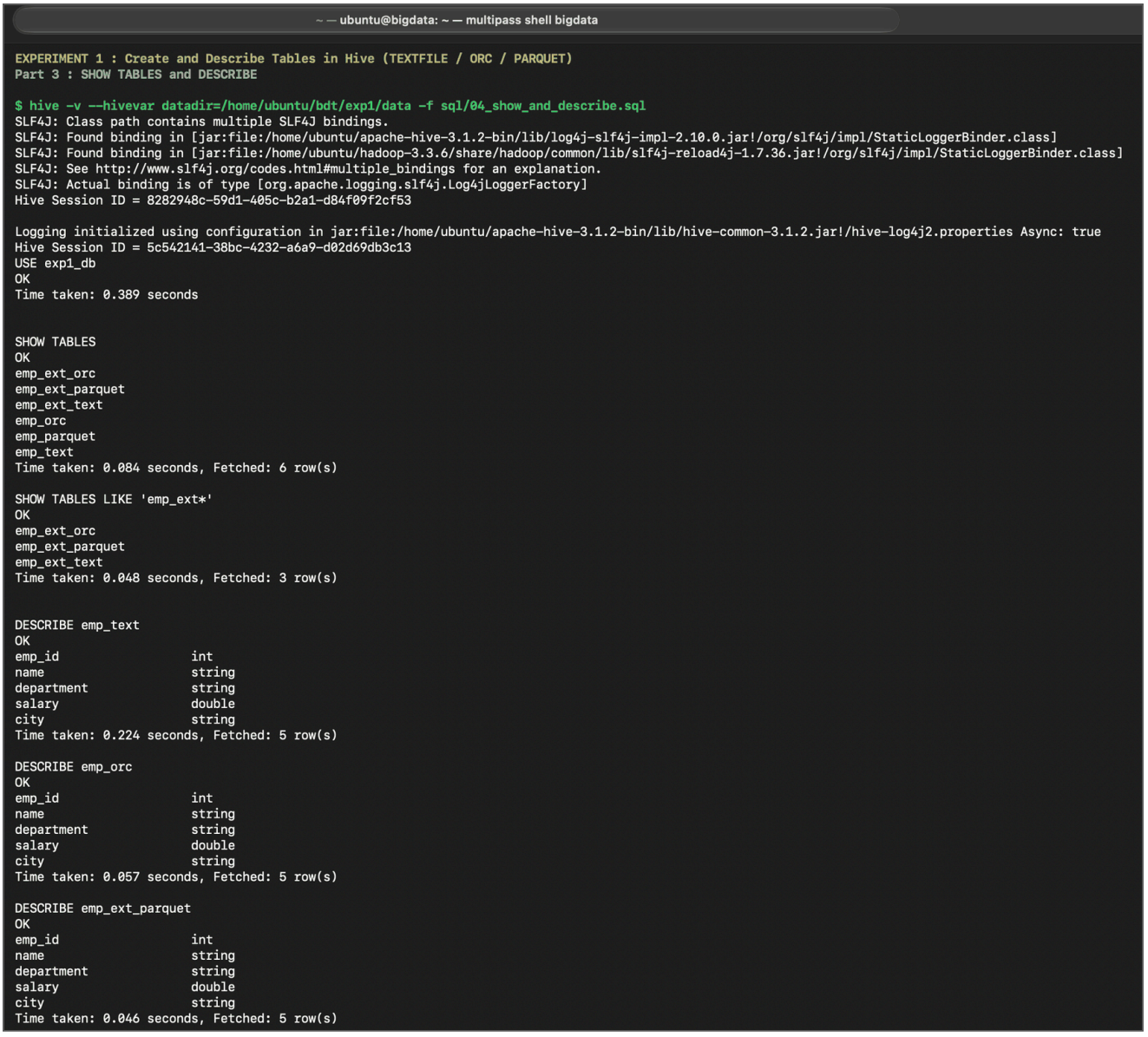


Figure 1.5: SHOW TABLES and DESCRIBE

`SHOW TABLES` lists all six tables; `SHOW TABLES LIKE 'emp_ext*'` filters the three external ones. `DESCRIBE` on a TEXTFILE, an ORC and a PARQUET table returns the same five columns and data types – the logical schema is independent of the storage format.

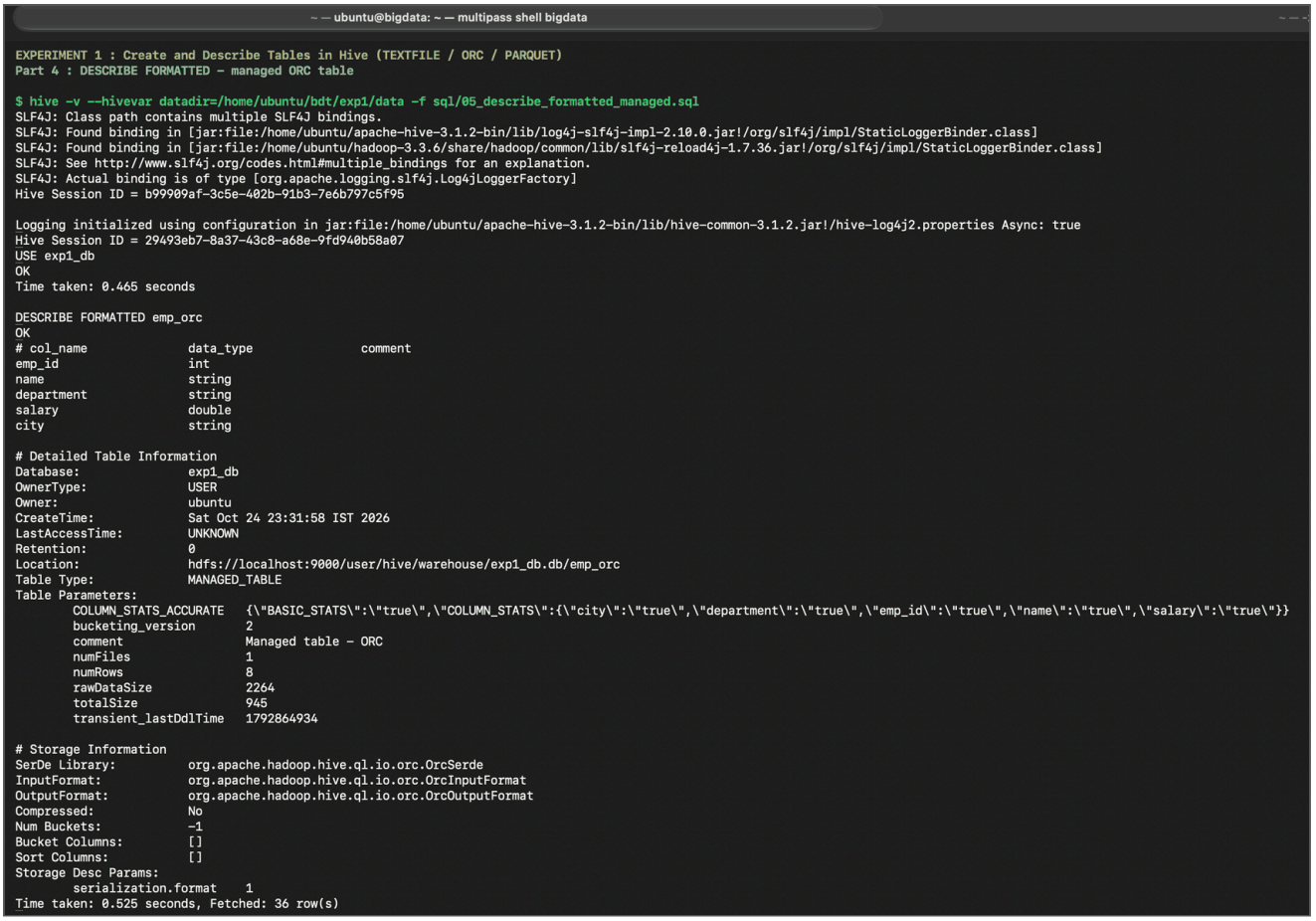


Figure 1.6: DESCRIBE FORMATTED of the managed ORC table

Key metadata: `Location: hdfs://localhost:9000/user/hive/warehouse/exp1_db.db/emp_orc`, `Table Type: MANAGED_TABLE`, `numRows 8`, and the storage classes `OrcSerde`, `OrcInputFormat` and `OrcOutputFormat`.

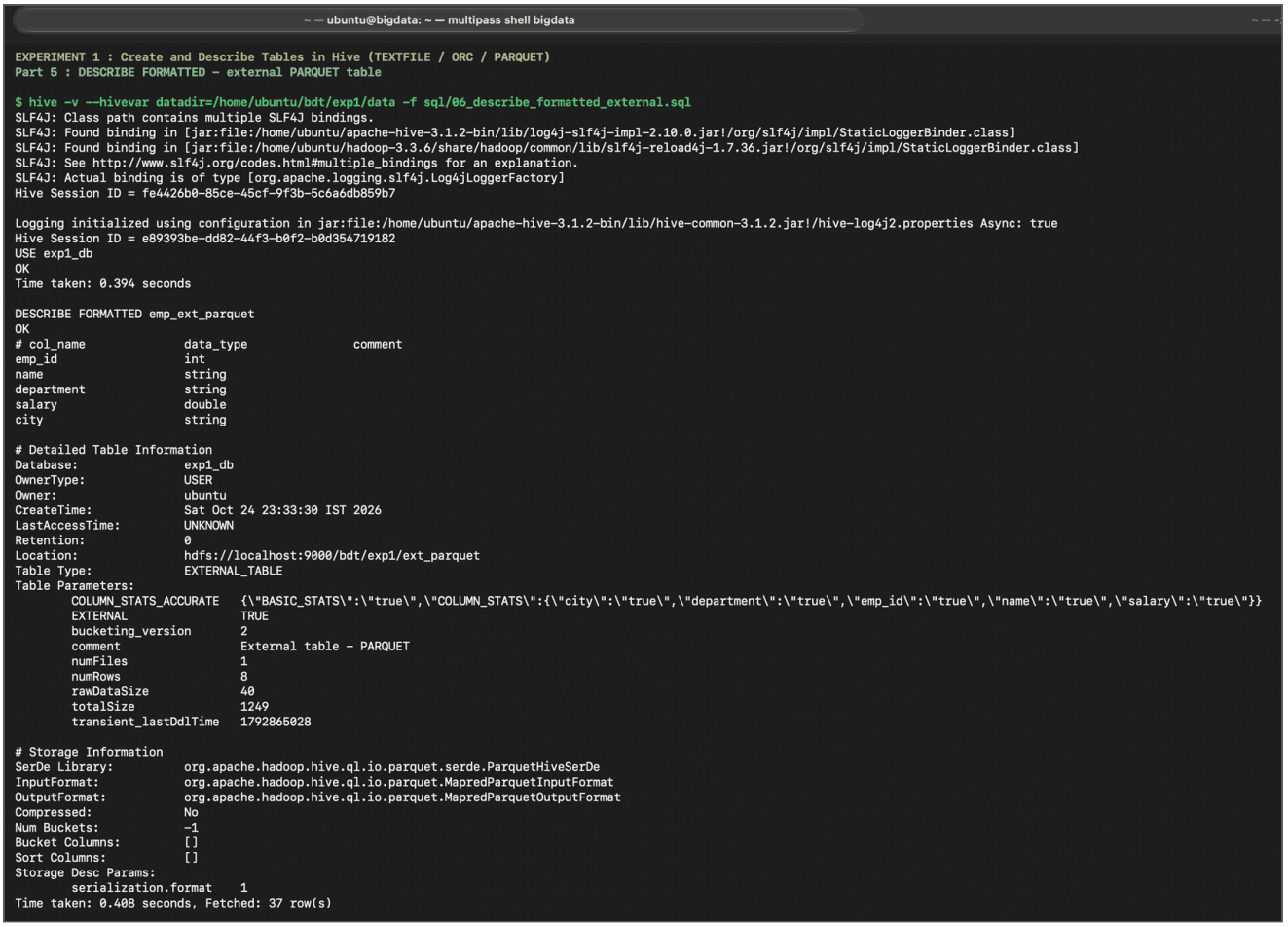


Figure 1.7: DESCRIBE FORMATTED of the external PARQUET table

In contrast to Figure 1.6: `Location: hdfs://localhost:9000/bdt/exp1/ext_parquet` (outside the warehouse), `Table Type: EXTERNAL_TABLE`, table parameter `EXTERNAL TRUE`, and the Parquet classes `ParquetHiveSerDe`, `MapredParquetInputFormat` and `MapredParquetOutputFormat`.

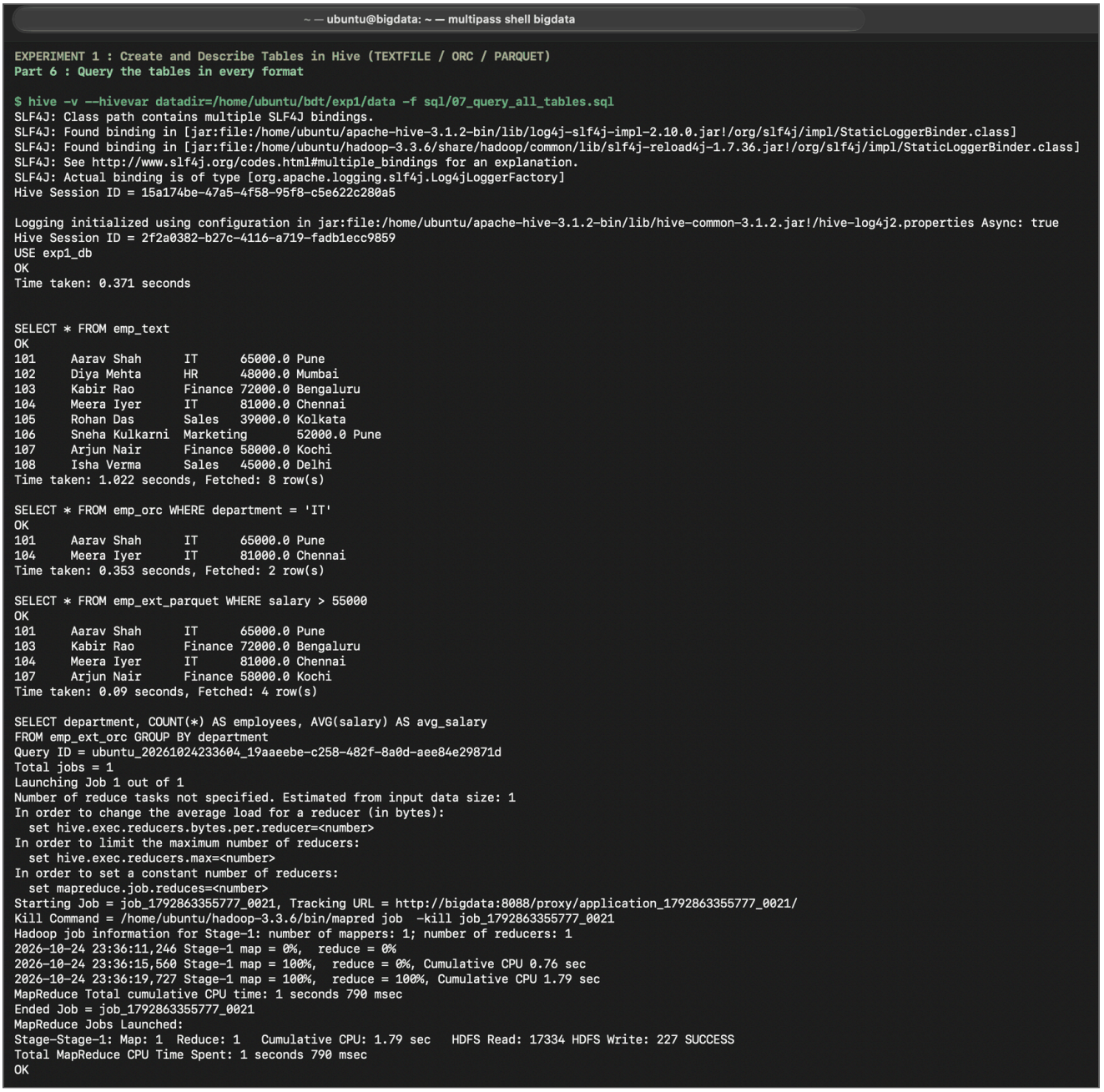


Figure 1.8: Querying the tables in every format

`SELECT *` on the TEXTFILE table returns all 8 rows, a filter on the ORC table returns the 2 IT employees, a filter on the external PARQUET table returns the 4 employees earning more than 55,000, and the GROUP BY on the external ORC table runs a MapReduce job that completes with `SUCCESS`. (The grouped result rows printed after this job are listed in Section G from the execution log.)

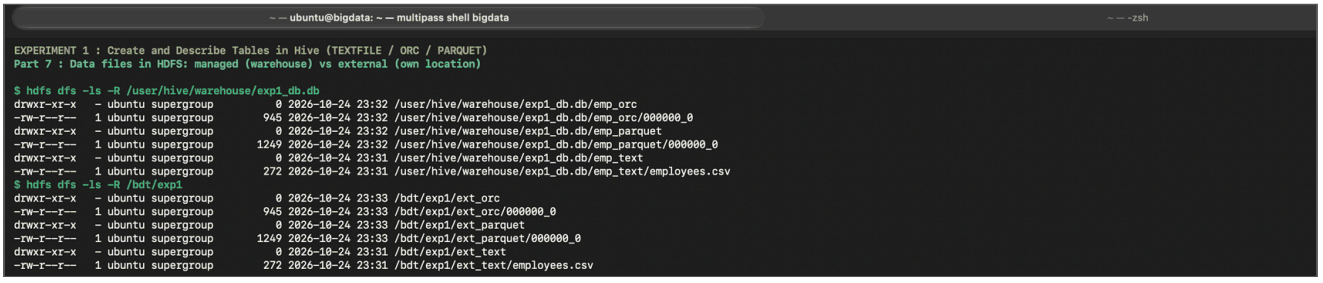


Figure 1.9: Data files in HDFS: managed vs external tables

Managed tables store their files under `/user/hive/warehouse/exp1_db.db/<table>/`, whereas the external tables use their own directories under `/bdt/exp1/`. File sizes: TEXTFILE 272 bytes, ORC 945 bytes, PARQUET 1249 bytes.

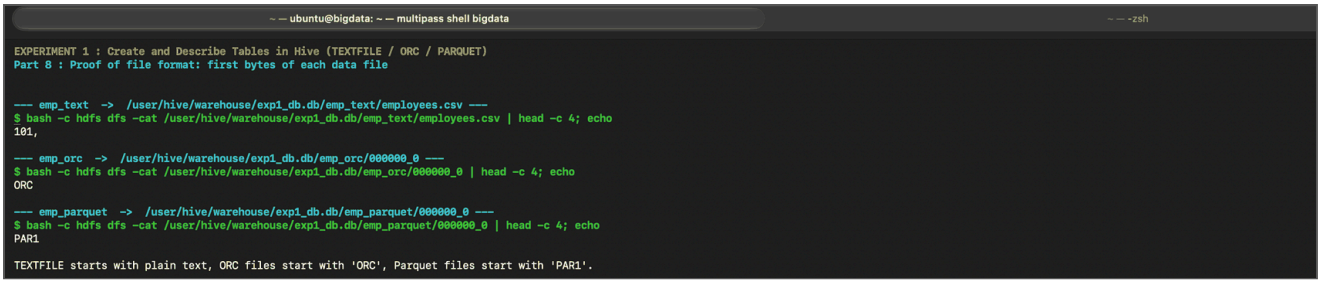


Figure 1.10: Proof of the physical file format

The first bytes of each data file: the TEXTFILE begins with plain text (`101,`), the ORC file with the magic bytes `ORC` and the PARQUET file with `PAR1` – confirming that Hive really wrote three different storage formats.

G. Output and Results

G.1 Tables created

Table	Type	Format	HDFS location	Data file (size)
emp_text	MANAGED	TEXTFILE	/user/hive/warehouse/exp1_db.db/emp_text	employees.csv (272 B)
emp_orc	MANAGED	ORC	/user/hive/warehouse/exp1_db.db/emp_orc	000000_0 (945 B)
emp_parquet	MANAGED	PARQUET	/user/hive/warehouse/exp1_db.db/emp_parquet	000000_0 (1249 B)
emp_ext_text	EXTERNAL	TEXTFILE	/bdt/exp1/ext_text	employees.csv (272 B)
emp_ext_orc	EXTERNAL	ORC	/bdt/exp1/ext_orc	000000_0 (945 B)
emp_ext_parquet	EXTERNAL	PARQUET	/bdt/exp1/ext_parquet	000000_0 (1249 B)

Source: Figures 1.5, 1.6, 1.7 and 1.9.

G.2 Query results

Query	Rows returned
`SELECT * FROM emp_text`	8 (all employees 101–108)
`SELECT * FROM emp_orc WHERE department = 'IT'`	2 – 101 Aarav Shah, 104 Meera Iyer
`SELECT * FROM emp_ext_parquet WHERE salary > 55000`	4 – 101, 103, 104, 107

Department	Employees	Average salary
Finance	2	65000.0
HR	1	48000.0
IT	2	73000.0
Marketing	1	52000.0
Sales	2	42000.0

The grouped result above was produced by `SELECT department, COUNT(*), AVG(salary) FROM emp_ext_orc GROUP BY department` and is taken from the execution log of the run (*Logs/*); the job itself is shown completing in Figure 1.8.

These results show that the same eight records are readable through every table, whatever its type (managed or external) and storage format (TEXTFILE, ORC, PARQUET), and that DESCRIBE / DESCRIBE FORMATTED expose the differences in location, table type and SerDe.

H. Result

> The objective was achieved. Three managed tables and three external tables were created in TEXTFILE, ORC and PARQUET formats and loaded with the same dataset. `SHOW TABLES` listed all six tables, `DESCRIBE` confirmed the identical column structure, and `DESCRIBE FORMATTED` confirmed the table type (MANAGED_TABLE / EXTERNAL_TABLE), HDFS location and storage format of each table. The HDFS listing and the file magic bytes (`ORC`, `PAR1`) independently verified the physical formats.

I. Observations and Troubleshooting

Observations

- **LOAD vs INSERT:** loading the CSV into a TEXTFILE table was a metadata/file-move operation that finished in about 0.3 s, while filling ORC and PARQUET tables required `INSERT ... SELECT` and a MapReduce job (about 17–18 s each) because the rows had to be rewritten in a columnar format.
- **File size:** for this tiny dataset the columnar files are larger (ORC 945 B, PARQUET 1249 B) than the 272-byte text file, because they carry footers, schema and column statistics. Their compression and column-pruning advantages only appear with large data.
- **Schema vs storage:** `DESCRIBE` output is identical for all formats; only `DESCRIBE FORMATTED` reveals the format-specific SerDe and Input/OutputFormat classes.
- **Statistics:** tables filled by `INSERT` show `numRows 8` and `COLUMN_STATS_ACCURATE` in their parameters because Hive gathers basic statistics automatically.
- `SHOW DATABASES` also lists databases created by other lab experiments on the same VM (`mydb`, `exp3_db`, `exp4_db`).
- The `SLF4J: Class path contains multiple SLF4J bindings` lines printed at every Hive start are a harmless warning caused by Hive and Hadoop both shipping a logging binding.

Problems encountered and solutions

Problem (observed during the trial run)	Cause	Solution
A query that counted rows of all six tables with `UNION ALL` launched one job with 6 map tasks; it stopped at `map = 67%` for over 20 minutes, and later commands failed with `Retrying connect to server: 0.0.0.0:8032`.	The ResourceManager (port 8032) had been killed: each Hadoop daemon defaulted to a ~1 GB Java heap, and together with Hive and the map tasks this exceeded the VM's 4 GB of RAM.	Limited daemon heaps (`HADOOP_HEAPSIZE_MAX=384m`) and the Hive client (`HADOOP_HEAPSIZE=768`), set YARN to 2 GB with at most 2 concurrent map tasks per job, added 2 GB of swap, restarted Hadoop, and replaced the 6-way UNION query with simpler queries.
After the stuck sessions were killed, `DROP DATABASE exp1_db CASCADE` waited indefinitely.	The killed Hive sessions left table locks in the metastore (DbTxnManager is enabled for the ACID experiments) and no housekeeping service was running to expire them.	Stopped Hive and removed the stale rows from the metastore's `HIVE_LOCKS` table with a small JDBC utility (`tools/clear_hive_locks.sh`); the runner now does this before each reset.

Experiment 4: Alter and Drop Table Operations

A. Experiment Details

Field	Details
Sub-experiment No.	4
Title	Alter and Drop Table Operations
Description / Learning Objective (bucket list)	Practice schema evolution with ALTER TABLE (rename, add/replace columns, change data type). Drop and recover tables using DROP and MSCK REPAIR TABLE.
Student	Adesh Patankar
Subject	Big Data Technology Lab (BDTL)
Tool / Technology	Apache Hive 3.1.2 on Apache Hadoop 3.3.6 (HDFS + YARN)

B. Aim

To practise **schema evolution** in Hive using `ALTER TABLE` – renaming a table, adding columns, renaming a column, changing a column's data type and replacing the full column list – and to study how `DROP TABLE` behaves for managed and external tables and how a dropped external, partitioned table can be **recovered** with `MSCK REPAIR TABLE`.

C. Theory

C.1 Schema evolution and schema-on-read

Hive keeps a table's schema in the Metastore and applies it only when the data is read (**schema-on-read**). Most `ALTER TABLE` statements therefore change only metadata and finish in a fraction of a second, without rewriting the files in HDFS. When the files are read later, Hive maps the fields of each record to the current column list **by position**; columns that do not exist in the old files are returned as `NULL`.

Statement	Purpose	Effect on data in HDFS
`ALTER TABLE t RENAME TO t2`	Rename a table	For a managed table the warehouse directory is also renamed (`.../t` → `.../t2`).
`ALTER TABLE t ADD COLUMNS (c type [COMMENT ...], ...)`	Append new columns at the end of the schema	None; existing rows show NULL for the new columns.
`ALTER TABLE t CHANGE [COLUMN] old new type [COMMENT ...] [FIRST|AFTER col]`	Rename a column and/or change its type, comment or position	None (metadata only). Only compatible type changes are allowed by default (e.g. INT → BIGINT → DOUBLE), controlled by `hive.metastore.disallow.incompatible.col.type.changes`.
`ALTER TABLE t REPLACE COLUMNS (c type, ...)`	Replace the whole column list (can remove columns)	None; only supported for tables with a native SerDe such as the delimited TEXTFILE SerDe.
`DROP TABLE t`	Remove a table	Managed: metadata and data files deleted. External: only metadata deleted, files stay.

C.2 Partitions and MSCK REPAIR TABLE

A partitioned table (`PARTITIONED BY (year INT)`) stores each partition in its own sub-directory named `column=value`, for example `/bdt/exp4/sales/year=2023`. The list of partitions is kept in the Metastore. If directories are created directly in HDFS (by `hdfs dfs`, another tool, or because a table was dropped and re-created), the Metastore does not know about them and queries return no rows. `MSCK REPAIR TABLE t` (metastore check) scans the table location, reports *Partitions not in metastore* and adds them, making the data visible again.

C.3 Recovering a dropped table

Because dropping an **external** table leaves its files in HDFS, the table can be recovered by re-running the same `CREATE EXTERNAL TABLE ... LOCATION` statement and then `MSCK REPAIR TABLE` to re-register all partition directories. A dropped **managed** table cannot be recovered this way, because its files are deleted together with the metadata.

D. Requirements

Item	Used in this experiment
Host machine	Apple MacBook Air (Apple Silicon, arm64) running macOS
Virtual machine	Multipass VM `bigdata` – Ubuntu 24.04 LTS (arm64), 2 vCPU, 4 GB RAM, 20 GB disk, 2 GB swap
Java	OpenJDK 8 (`/usr/lib/jvm/java-8-openjdk-arm64`)
Hadoop	Apache Hadoop 3.3.6, pseudo-distributed (single node): HDFS at `hdfs://localhost:9000` + YARN; daemons NameNode, DataNode, SecondaryNameNode, ResourceManager, NodeManager
Hive	Apache Hive 3.1.2, Hive CLI, execution engine MapReduce (Hive-on-MR), embedded Apache Derby metastore, warehouse `/user/hive/warehouse`
Scripts	Bash runner scripts + HiveQL files (supplied separately in *Scripts/*); each HiveQL file is run with `hive -v -f` so that every statement is echoed before its output
Datasets	`students.csv` (6 records: id, name, marks); `sales_2023.csv`, `sales_2024.csv`, `sales_2025.csv` (id, product, amount)
HDFS paths	Managed table: `/user/hive/warehouse/exp4_db.db/`; external partitioned table: `/bdt/exp4/sales/year=YYYY/`
Pre-requisites	Hadoop daemons running, Hive metastore initialised, database `exp4_db` created by the script

E. Procedure and Implementation

The experiment was executed with `bdt/exp4/run_exp4.sh`, which resets the experiment (drops `exp4_db` and `/bdt/exp4`) and then runs eight parts. HiveQL files are executed with `hive -v -f` so that each statement appears in the terminal before its result.

Input data

exp4/data/students.csv

```
1,Aarav,82
2,Diya,68
3,Kabir,91
4,Meera,74
5,Rohan,79
6,Sneha,88
```

exp4/data/sales_2023.csv | sales_2024.csv | sales_2025.csv

```
1,Laptop,55000
2,Mouse,700
3,Keyboard,1500

4,Monitor,12000
5,Laptop,61000
6,Headphones,2500

7,Tablet,30000
8,Printer,9000
```

Step 1 – Create and load the starting table

exp4/sql/01_create_table.sql

```
-- Experiment 4 | Step 1 : create the starting table
CREATE DATABASE IF NOT EXISTS exp4_db;
USE exp4_db;

CREATE TABLE IF NOT EXISTS student (
  id    INT,
  name  STRING,
  marks INT)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE;

LOAD DATA LOCAL INPATH '${hivevar:datadir}/students.csv' OVERWRITE INTO TABLE student;

DESCRIBE student;
SELECT * FROM student;
```

Step 2 – Rename the table

exp4/sql/02_rename_table.sql

```
-- Experiment 4 | Step 2 : ALTER TABLE ... RENAME TO
USE exp4_db;
SHOW TABLES;

ALTER TABLE student RENAME TO student_info;

SHOW TABLES;
SELECT * FROM student_info;
```

Step 3 – Add columns

exp4/sql/03_add_columns.sql

```
-- Experiment 4 | Step 3 : ALTER TABLE ... ADD COLUMNS (schema evolution)
USE exp4_db;

ALTER TABLE student_info ADD COLUMNS (
  email STRING COMMENT 'student e-mail id',
  city  STRING COMMENT 'home city');

DESCRIBE student_info;
-- existing rows show NULL for the new columns
SELECT * FROM student_info;
```

Step 4 – Rename a column and change data types

exp4/sql/04_change_column.sql

```
-- Experiment 4 | Step 4 : ALTER TABLE ... CHANGE (rename a column / change data type)
USE exp4_db;

-- change data type: INT -> BIGINT (a widening, compatible change)
ALTER TABLE student_info CHANGE marks marks BIGINT;

-- rename a column (name -> student_name), keeping its type
ALTER TABLE student_info CHANGE name student_name STRING COMMENT 'full name';

-- change data type: BIGINT -> DOUBLE
ALTER TABLE student_info CHANGE marks marks DOUBLE COMMENT 'marks out of 100';
```

```
DESCRIBE student_info;
SELECT * FROM student_info;
```

Step 5 – Replace the column list

exp4/sql/05_replace_columns.sql

```
-- Experiment 4 | Step 5 : ALTER TABLE ... REPLACE COLUMNS
-- REPLACE COLUMNS swaps the whole column list (here it removes email and city)
USE exp4_db;

ALTER TABLE student_info REPLACE COLUMNS (
  id           INT,
  student_name STRING,
  marks        DOUBLE);

DESCRIBE student_info;
SELECT * FROM student_info;
```

Step 6 – Drop the managed table

exp4/sql/06_drop_managed.sql

```
-- Experiment 4 | Step 6 : DROP a MANAGED table (metadata AND data are deleted)
USE exp4_db;
SHOW TABLES;
DROP TABLE student_info;
SHOW TABLES;
```

Shell command

```
hdfs dfs -ls -R /user/hive/warehouse/exp4_db.db   # run after Steps 1, 2 and 6
```

Step 7 – Create partition folders in HDFS and an external partitioned table over them

Shell commands

```
hdfs dfs -mkdir -p /bdt/exp4/sales/year=2023 /bdt/exp4/sales/year=2024
hdfs dfs -put -f data/sales_2023.csv /bdt/exp4/sales/year=2023/
hdfs dfs -put -f data/sales_2024.csv /bdt/exp4/sales/year=2024/
hdfs dfs -ls -R /bdt/exp4/sales
```

exp4/sql/07_external_partitioned.sql

```
-- Experiment 4 | Step 7 : EXTERNAL partitioned table over existing HDFS folders
USE exp4_db;

CREATE EXTERNAL TABLE IF NOT EXISTS sales (
  id      INT,
  product STRING,
  amount  DOUBLE)
PARTITIONED BY (year INT)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/bdt/exp4/sales';

-- the folders year=2023 and year=2024 exist in HDFS, but the metastore does not know them yet
SHOW PARTITIONS sales;
SELECT COUNT(*) AS rows_visible FROM sales;
```

Step 8 – Register the partitions with MSCK REPAIR TABLE

exp4/sql/08_msck_repair.sql

```
-- Experiment 4 | Step 8 : MSCK REPAIR TABLE registers the partitions found in HDFS
USE exp4_db;

MSCK REPAIR TABLE sales;

SHOW PARTITIONS sales;
SELECT * FROM sales;
```

Step 9 – Drop the external table; new data arrives while it is dropped

exp4/sql/09_drop_external.sql

```
-- Experiment 4 | Step 9 : DROP the EXTERNAL table (only metadata is removed)
USE exp4_db;
DROP TABLE sales;
SHOW TABLES;
```

Shell commands

```
hdfs dfs -mkdir -p /bdt/exp4/sales/year=2025
hdfs dfs -put -f data/sales_2025.csv /bdt/exp4/sales/year=2025/
hdfs dfs -ls -R /bdt/exp4/sales
```

Step 10 – Recover the table

exp4/sql/10_recover_table.sql

```
-- Experiment 4 | Step 10 : RECOVER the dropped table: re-create the definition,
-- then MSCK REPAIR TABLE re-attaches every partition still in HDFS
-- (including year=2025, which was added directly to HDFS after the drop)
USE exp4_db;

CREATE EXTERNAL TABLE IF NOT EXISTS sales (
  id      INT,
  product STRING,
  amount  DOUBLE)
PARTITIONED BY (year INT)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/bdt/exp4/sales';

SHOW PARTITIONS sales;
MSCK REPAIR TABLE sales;
SHOW PARTITIONS sales;

SELECT * FROM sales;
SELECT year, COUNT(*) AS orders, SUM(amount) AS revenue FROM sales GROUP BY year;
```

F. Execution Screenshots

All screenshots were captured from the Terminal of the Ubuntu VM (`ubuntu@bigdata`) while `run_exp4.sh` was executing. Only empty margins were trimmed for printing.

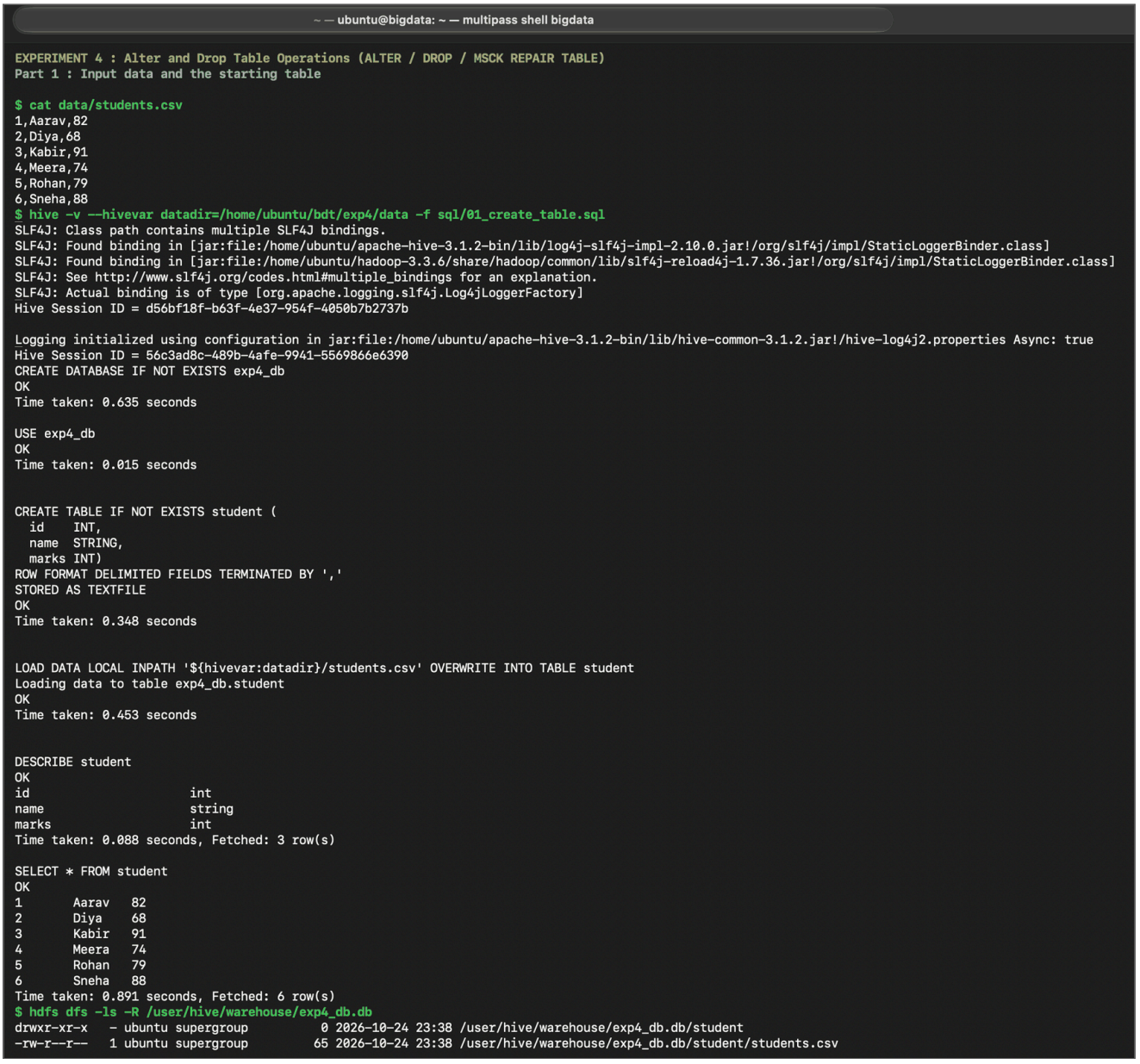


Figure 4.1: Input data, creation and loading of the starting table `student`

`students.csv` (6 rows) is loaded into the managed TEXTFILE table `student(id INT, name STRING, marks INT)`. `DESCRIBE` shows the three columns, `SELECT *` returns the 6 rows, and the HDFS listing shows the file at `/user/hive/warehouse/exp4_db.db/student/students.csv`.

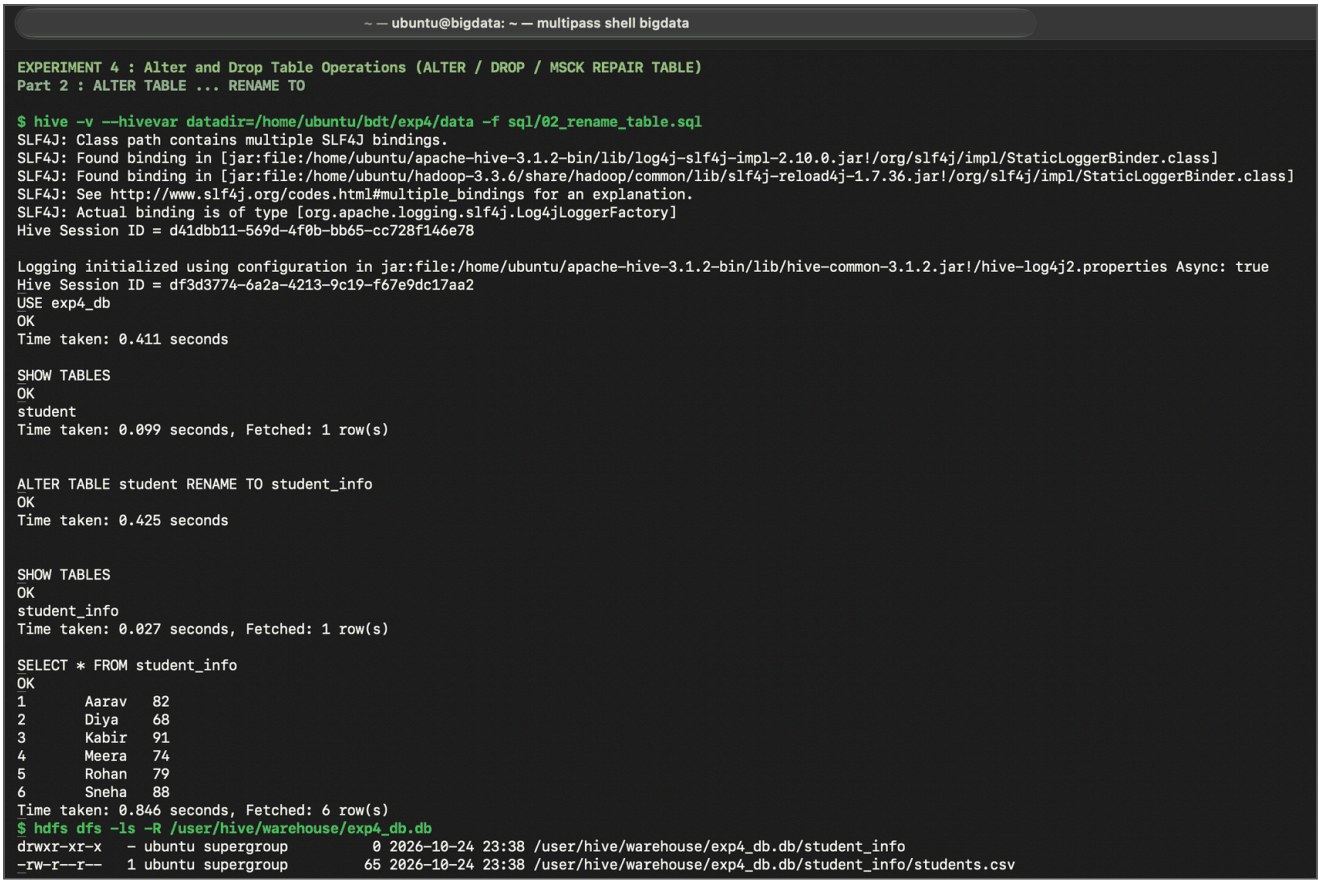


Figure 4.2: ALTER TABLE ... RENAME TO

`SHOW TABLES` lists `student` before and `student_info` after the rename; the data is still readable, and the HDFS listing shows the warehouse directory itself was renamed to `.../student_info`.

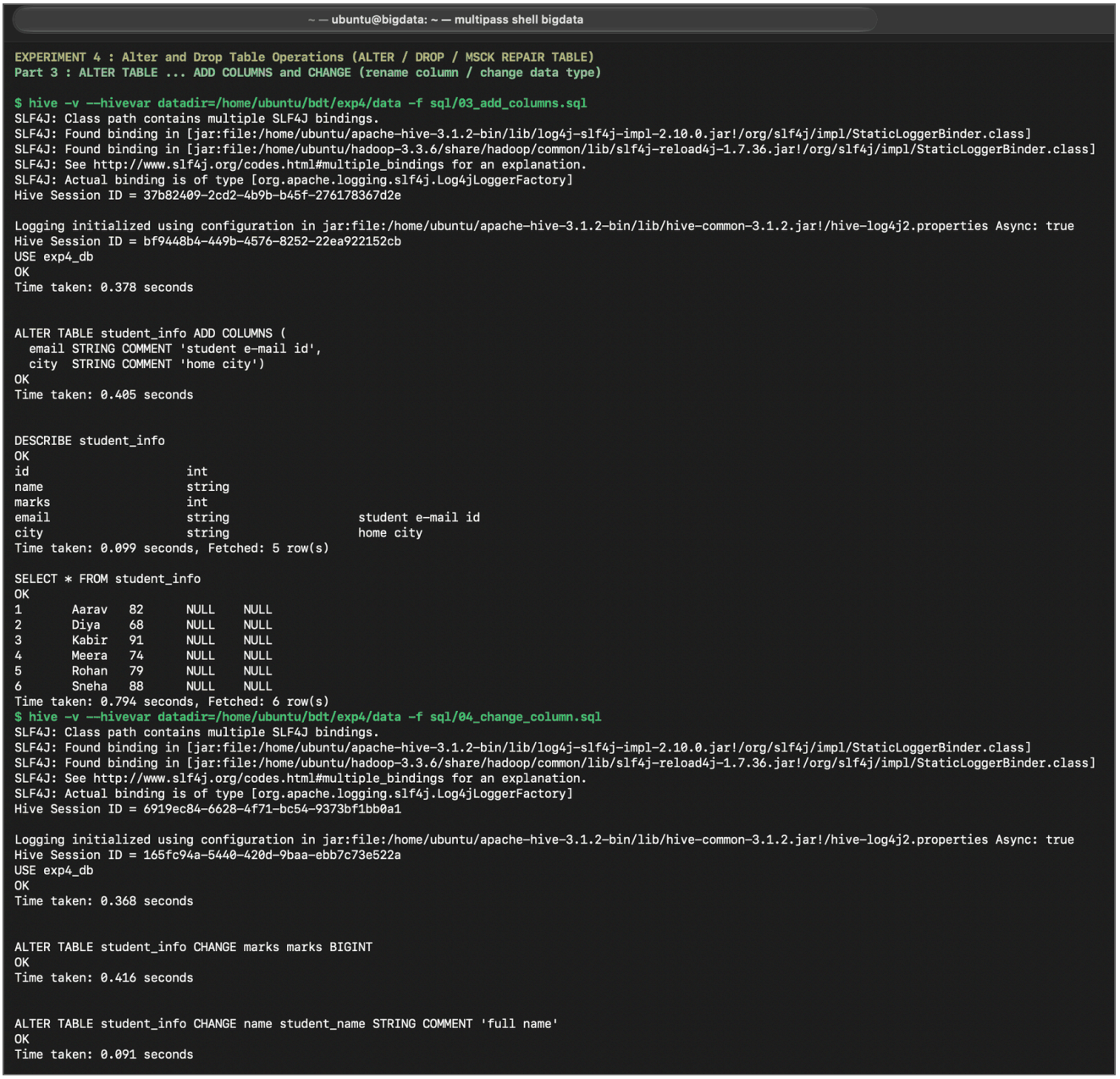


Figure 4.3: ALTER TABLE ... ADD COLUMNS, followed by the start of the CHANGE statements

Columns `email` and `city` (with comments) are added. `DESCRIBE` now lists five columns and `SELECT *` shows `NULL` in the new columns for every existing row, because the old file has only three fields. The lower part shows the first two `CHANGE` statements executing.

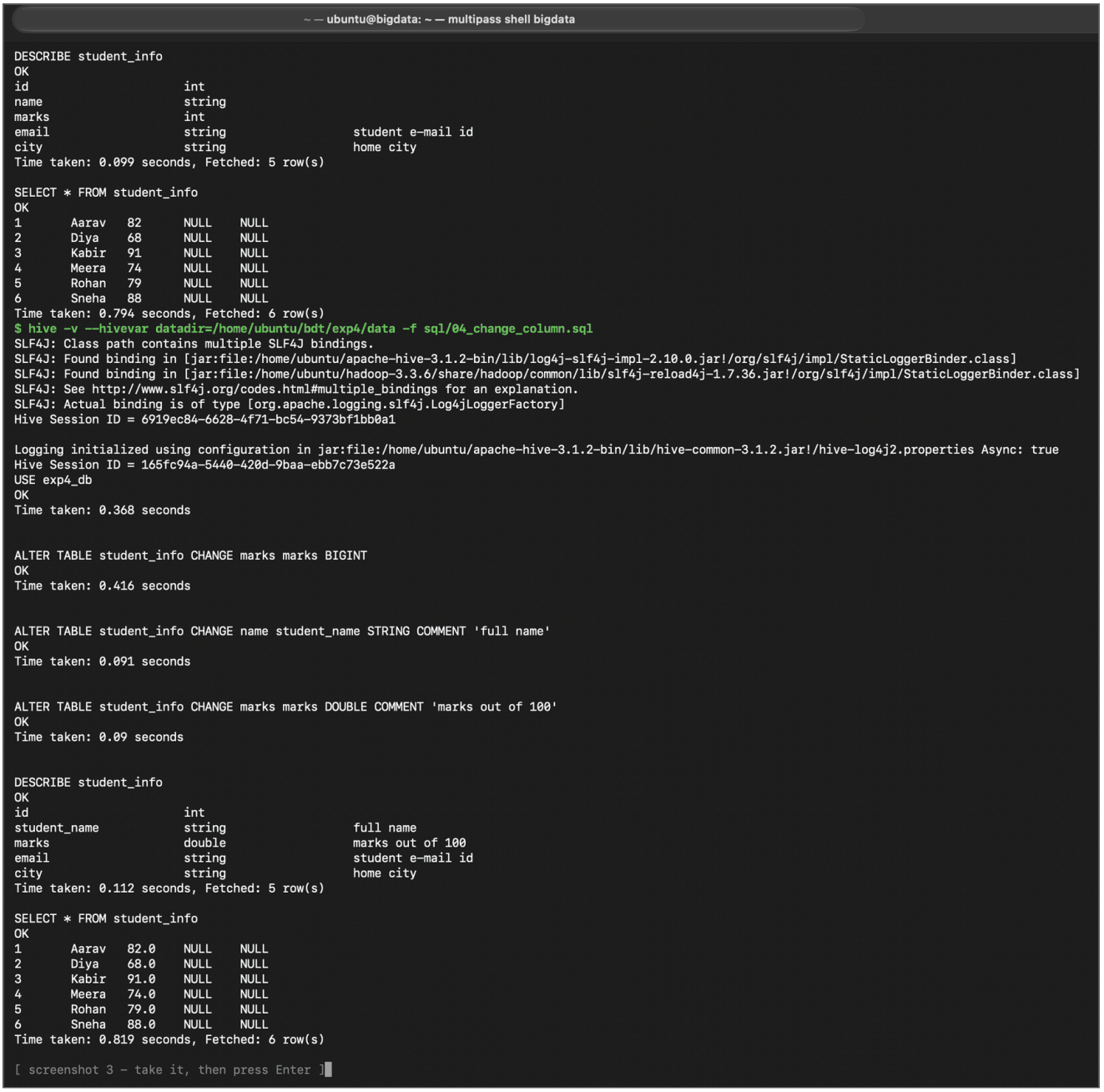


Figure 4.4: ALTER TABLE … CHANGE: data-type changes and column rename

`marks` is changed INT → BIGINT → DOUBLE and `name` is renamed to `student_name`. The final `DESCRIBE` shows `student_name string` and `marks double` with their new comments, and the data is now displayed as `82.0`, `68.0`, … – the same file read with the new type.

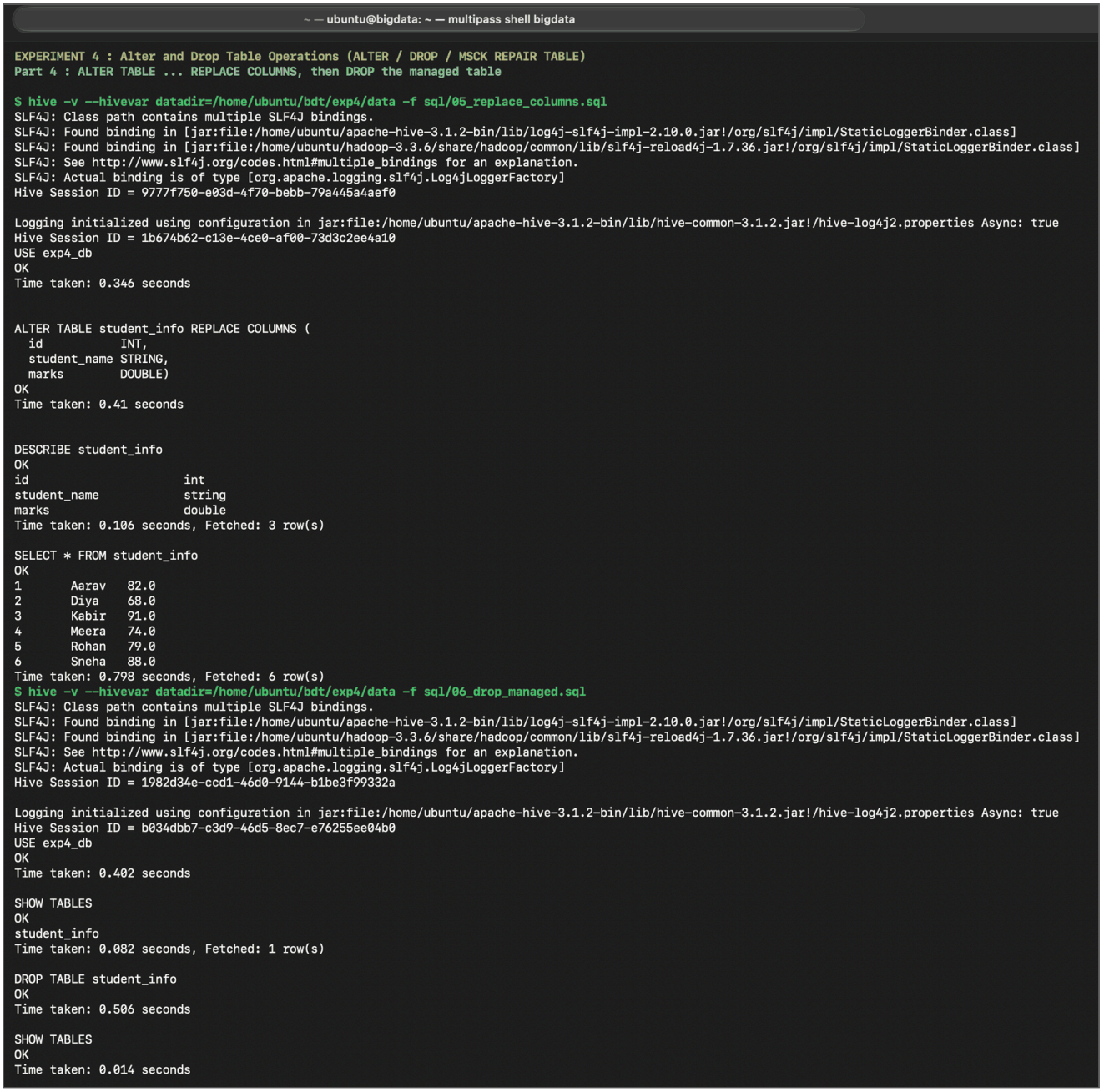


Figure 4.5: ALTER TABLE ... REPLACE COLUMNS and DROP of the managed table

`REPLACE COLUMNS (id INT, student_name STRING, marks DOUBLE)` removes `email` and `city`; `DESCRIBE` shows three columns again. Then `DROP TABLE student_info` removes the managed table – the second `SHOW TABLES` returns no rows.

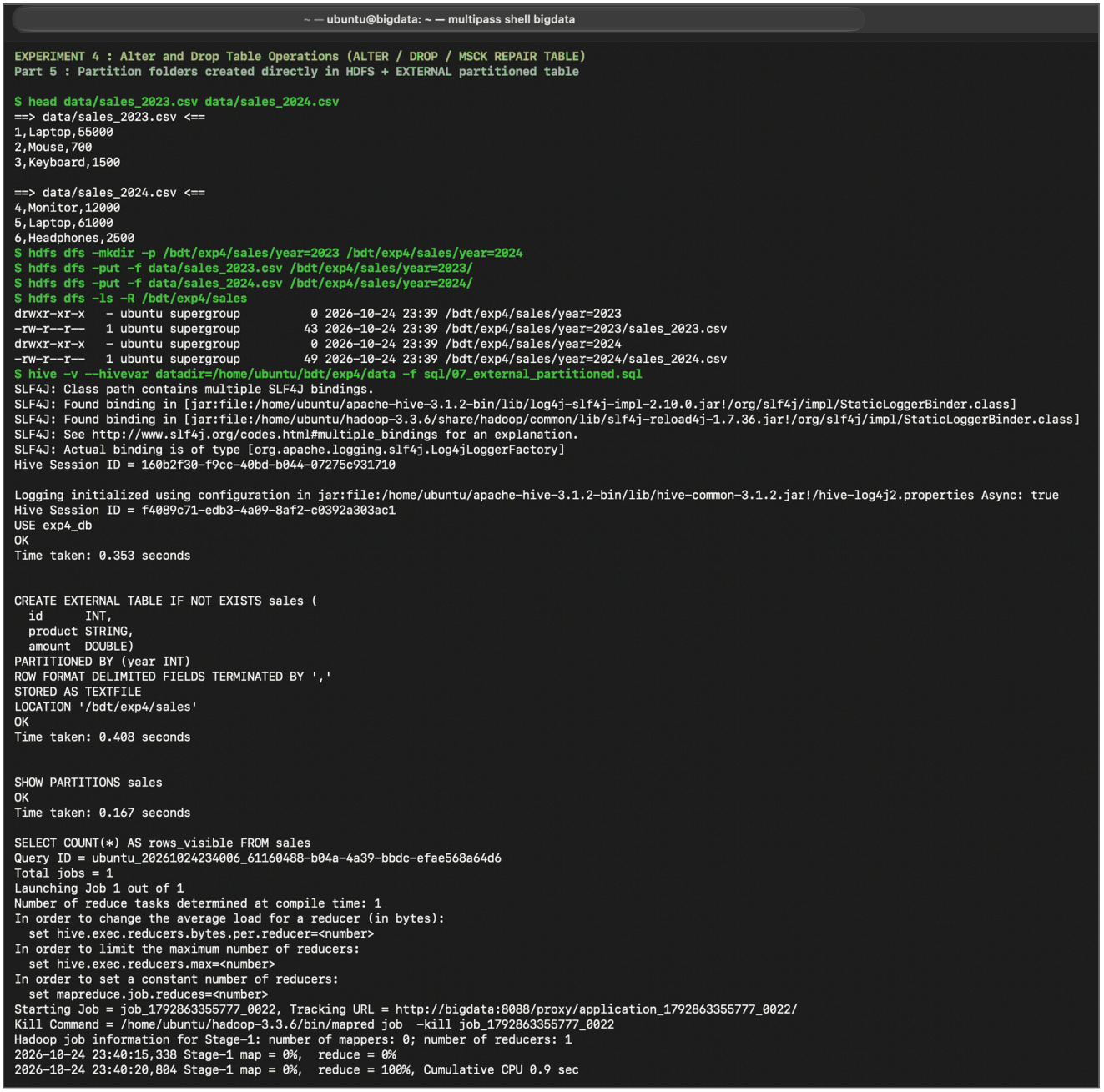


Figure 4.6: Partition folders created directly in HDFS and an external partitioned table over them

Directories `year=2023` and `year=2024` with one CSV each are created with `hdfs dfs`. The external table `sales` is created with `PARTITIONED BY (year INT)` and `LOCATION '/bdt/exp4/sales'`; `SHOW PARTITIONS sales` returns nothing, because the Metastore does not yet know these folders. (The screenshot ends while the row-count job is still running.)

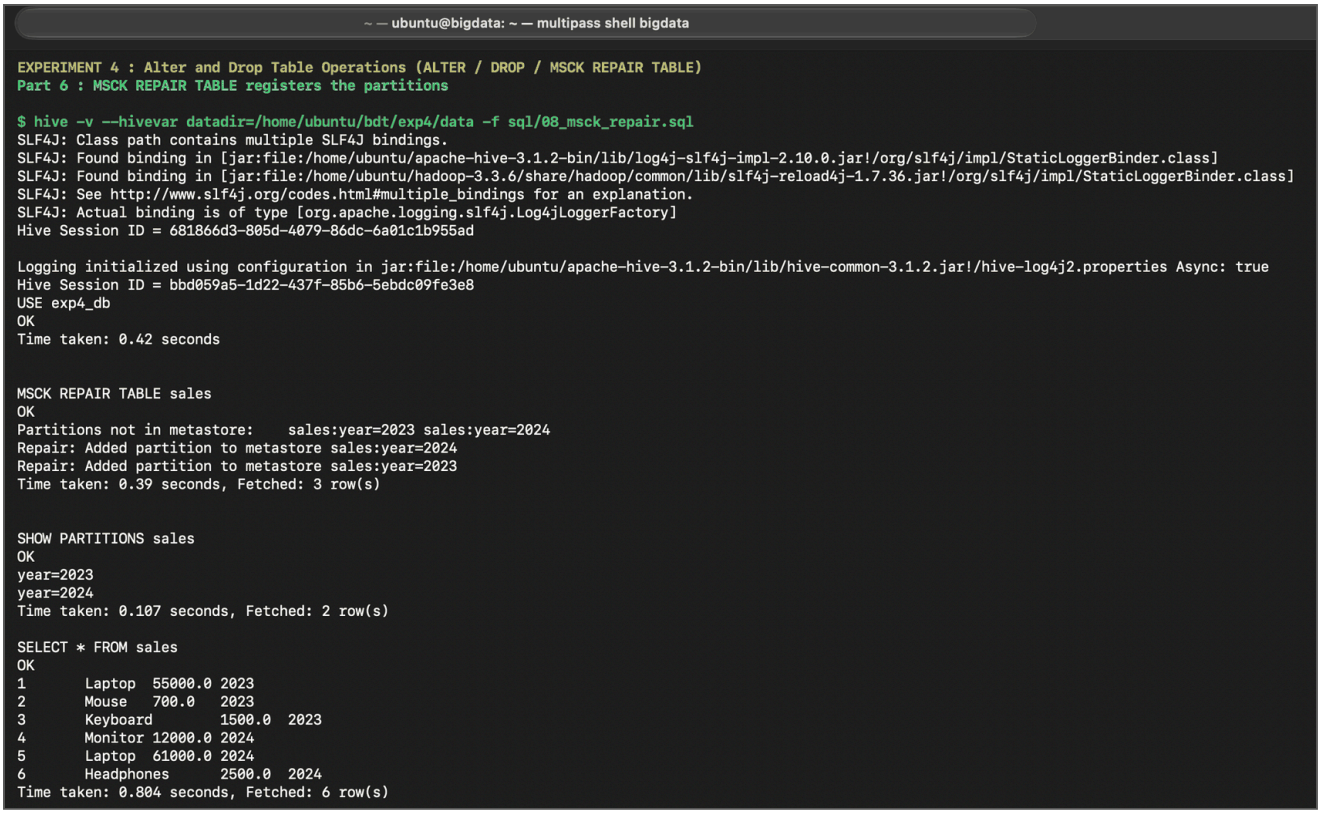


Figure 4.7: MSCK REPAIR TABLE registers the partitions

MSCK reports `Partitions not in metastore: sales:year=2023 sales:year=2024` and adds both. `SHOW PARTITIONS` now lists them and `SELECT *` returns all 6 sales records with their `year`.

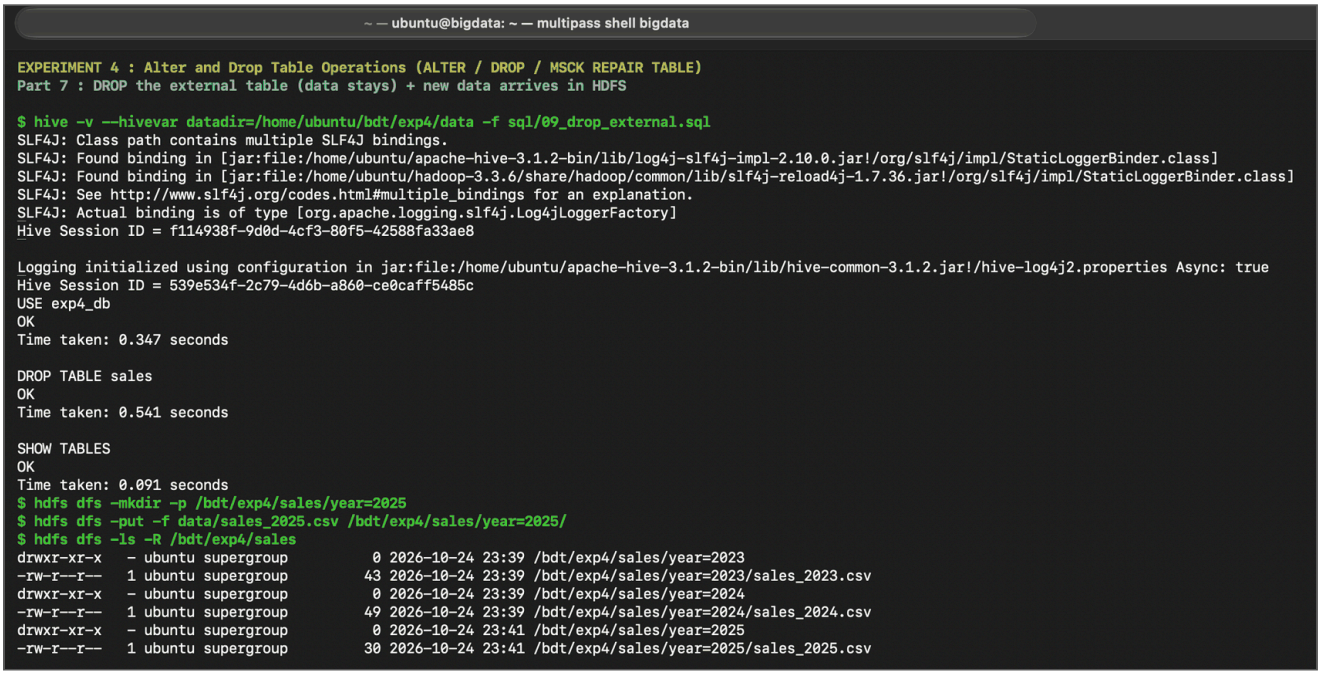


Figure 4.8: DROP of the external table – the data stays in HDFS – and new data arrives

`DROP TABLE sales` succeeds and `SHOW TABLES` is empty, yet the HDFS listing still shows the 2023 and 2024 files. A new partition folder `year=2025` is then added directly in HDFS while the table does not exist.

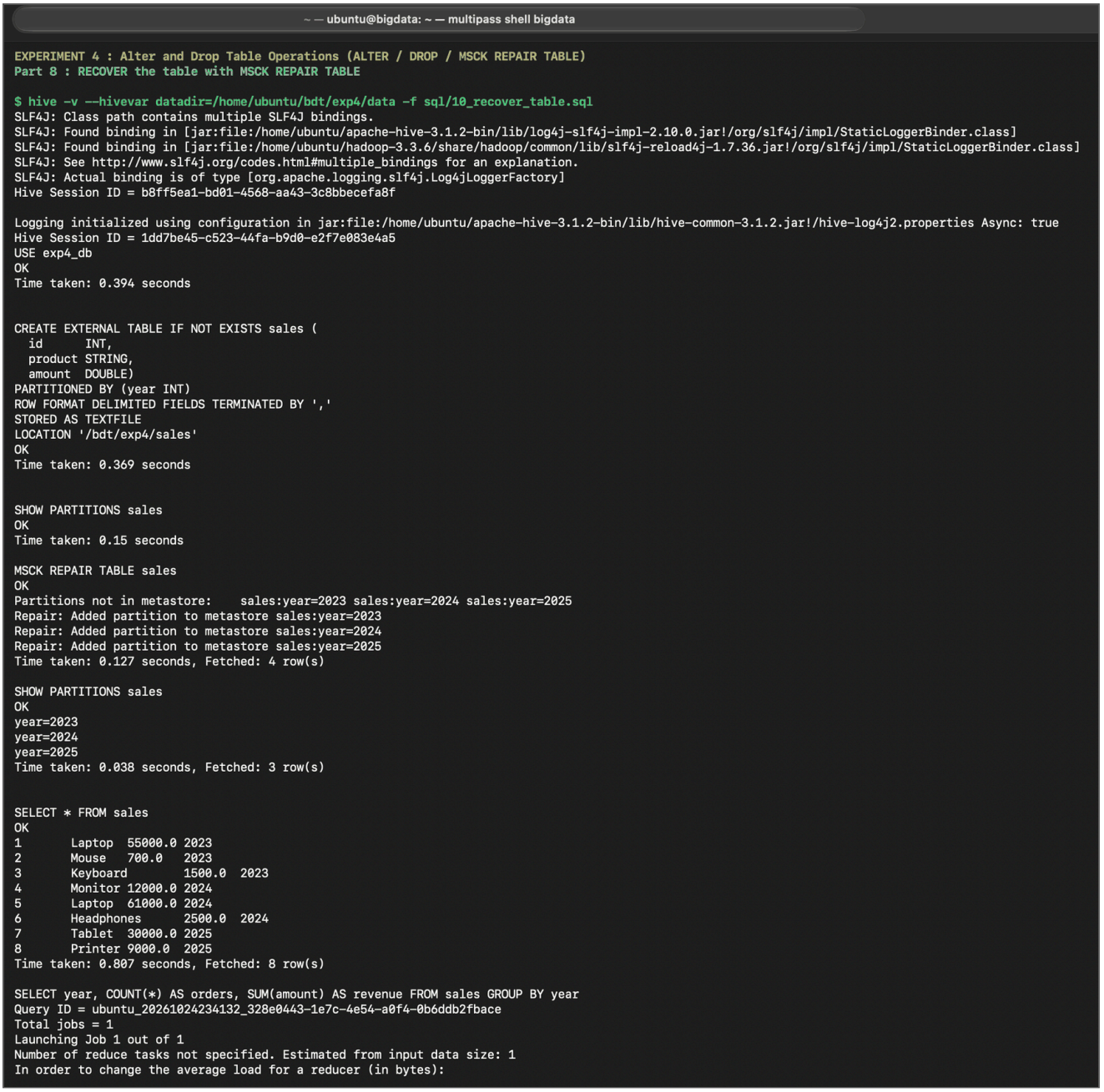


Figure 4.9: Recovering the dropped table with CREATE EXTERNAL TABLE + MSCK REPAIR TABLE

The table definition is re-created; `SHOW PARTITIONS` is empty at first. `MSCK REPAIR TABLE sales` finds and adds `year=2023`, `year=2024` and `year=2025`; `SELECT *` returns all 8 records.

```
SELECT year, COUNT(*) AS orders, SUM(amount) AS revenue FROM sales GROUP BY year
Query ID = ubuntu_20261024234132_328e0443-1e7c-4e54-a0f4-0b6ddb2fbace
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1792863355777_0023, Tracking URL = http://bigdata:8088/proxy/application_1792863355777_0023/
Kill Command = /home/ubuntu/hadoop-3.3.6/bin/mapred job  -kill job_1792863355777_0023
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-10-24 23:41:37,724 Stage-1 map = 0%,  reduce = 0%
2026-10-24 23:41:42,012 Stage-1 map = 100%,  reduce = 0%, Cumulative CPU 0.85 sec
2026-10-24 23:41:47,306 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 1.69 sec
MapReduce Total cumulative CPU time: 1 seconds 690 msec
Ended Job = job_1792863355777_0023
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 1.69 sec   HDFS Read: 15454 HDFS Write: 168 SUCCESS
Total MapReduce CPU Time Spent: 1 seconds 690 msec
OK
2023    3       57200.0
2024    3       75500.0
2025    2       39000.0
Time taken: 16.428 seconds, Fetched: 3 row(s)
```

Figure 4.10: Aggregation over the recovered table

`SELECT year, COUNT(*), SUM(amount) FROM sales GROUP BY year` runs a MapReduce job (`SUCCESS`) and returns 2023: 3 orders / 57200.0, 2024: 3 / 75500.0, 2025: 2 / 39000.0.

G. Output and Results

G.1 Schema evolution of the student table

After operation	Table name	Columns (DESCRIBE)	Sample row
CREATE + LOAD	student	id int, name string, marks int	1 Aarav 82
RENAME TO	student_info	id int, name string, marks int	1 Aarav 82
ADD COLUMNS	student_info	id, name, marks int, email string, city string	1 Aarav 82 NULL NULL
CHANGE (type + rename)	student_info	id int, student_name string, marks double, email, city	1 Aarav 82.0 NULL NULL
REPLACE COLUMNS	student_info	id int, student_name string, marks double	1 Aarav 82.0
DROP TABLE	–	table removed (managed: data deleted)	–

Source: Figures 4.1–4.5.

G.2 Drop and recovery of the external partitioned table

Stage	SHOW PARTITIONS	Rows returned	Files in HDFS
External table created over existing folders	(none)	–	year=2023, year=2024
After MSCK REPAIR TABLE	year=2023, year=2024	6	year=2023, year=2024
After DROP TABLE (+ 2025 data added)	table does not exist	–	year=2023, year=2024, year=2025
Re-created + MSCK REPAIR TABLE	year=2023, year=2024, year=2025	8	year=2023, year=2024, year=2025

year	orders	revenue
2023	3	57200.0
2024	3	75500.0
2025	2	39000.0

Source: Figures 4.6–4.10. The recovered table contains all original rows plus the partition that was added while the table was dropped, which demonstrates that MSCK REPAIR TABLE rebuilds the partition metadata from the directories present in HDFS.

H. Result

> The objective was achieved. The table was renamed with `RENAME TO`, extended with `ADD COLUMNS`, a column was renamed and its type changed INT → BIGINT → DOUBLE with `CHANGE`, and the column list was reduced with `REPLACE COLUMNS` – each change verified with `DESCRIBE` and `SELECT`. Dropping the managed table removed it, while dropping the external partitioned table left its data in HDFS; the table was then recovered with `CREATE EXTERNAL TABLE` + `MSCK REPAIR TABLE`, which restored all three partitions and 8 rows.

I. Observations and Troubleshooting

Observations

- All ALTER TABLE statements completed in about 0.1–0.4 s and launched no MapReduce job: they change only Metastore metadata. The data file `students.csv` (65 bytes) was never rewritten.
- Because the schema is applied on read, the same file was displayed as `82` (INT) and later as `82.0` (DOUBLE), and the added columns appeared as `NULL`.
- `RENAME TO` on a managed table also renamed its HDFS directory (Figure 4.2).
- Widening type changes (INT → BIGINT → DOUBLE) were accepted. Narrowing or incompatible changes (for example STRING → INT) are rejected by default by `hive.metastore.disallow.incompatible.col.type.changes=true`; this was not exercised in the run.
- Creating partition folders directly in HDFS is not enough for Hive to see the data; `SHOW PARTITIONS` stays empty until `MSCK REPAIR TABLE` (or `ALTER TABLE ... ADD PARTITION`) updates the Metastore.

- The ORDER in which MSCK reports the added partitions can vary between runs (Figure 4.7 lists 2024 before 2023).

Problems encountered and solutions

Problem	Cause	Solution
During the first trial of the experiments a MapReduce job froze at `map = 67%` and YARN commands failed with `Retrying connect to server: 0.0.0.0:8032`.	The ResourceManager was killed for lack of memory in the 4 GB VM (each Hadoop daemon defaulted to a ~1 GB heap).	Reduced daemon/Hive heap sizes, limited YARN to 2 GB with at most 2 concurrent map tasks, added 2 GB swap and restarted Hadoop. Experiment 4 then ran without errors.
Resetting a database after force-stopped Hive sessions hung indefinitely.	Stale table locks left in the Metastore's lock table by the killed sessions.	Removed the stale locks with `tools/clear_hive_locks.sh` before re-running.